hashlock.

Security Audit

Frenly (dApp)

August 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

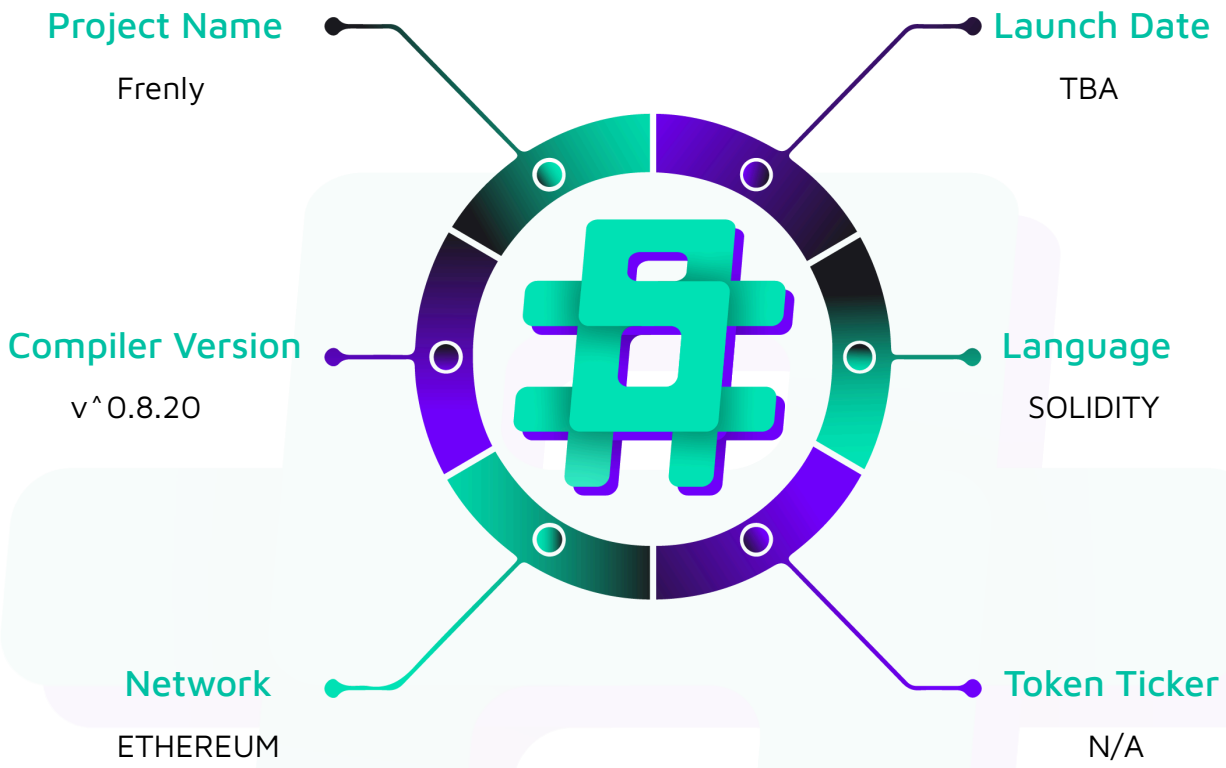
The Frenly team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

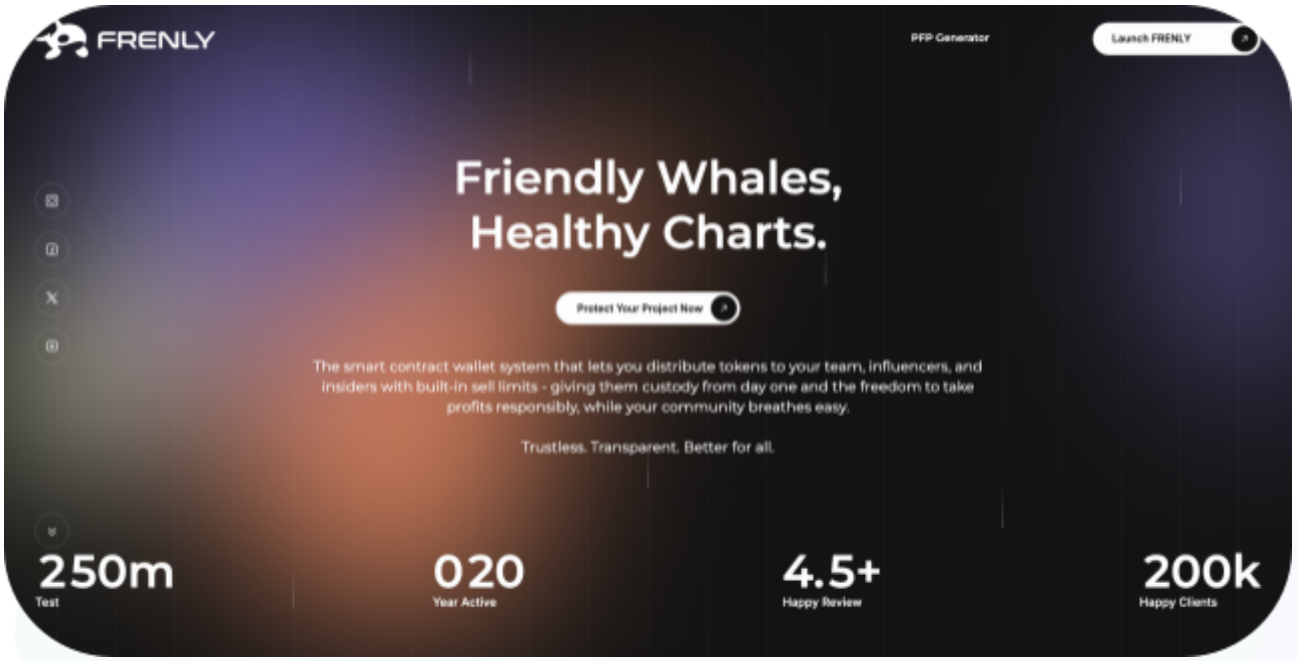
FRENLY is a smart-contract–based wallet solution that helps crypto projects distribute tokens to team members, influencers, or partners in a secure and market-friendly way. It allows projects to maintain custody from day one while enforcing automatic sell limits—both per transaction and per day—to prevent sudden dumps and protect price stability. Compatible with networks like Ethereum, Polygon, and Arbitrum (with Solana coming soon), FRENLY offers flexibility through its built-in DEX and integration with PoolParty for full liquidity when needed, making it an ideal tool for responsible token distribution.

Project Name: Frenly
Project Type: dApp
Compiler Version: ^0.8.20
Website: https://www.getfrenly.com/
Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Frenly project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Frenly Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**August, 2025**
Contract 1	Factory.sol
Contract 2	Vault.sol
Contract 3	ChainAddressRegistry.sol
Contract 4	ENSConfig.sol
Contract 5	ENSManagementLib.sol
Contract 6	NameGenerationLib.sol
Contract 7	VaultDeploymentLib.sol
Audited GitHub Commit Hash	b83b9ed38e1493fac17bfe0b3b4ea9407f36666f
Fix Review GitHub Commit Hash	41611e0b91caa11e201f1819720bc2e60ec407a9

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerabilities

4 Medium severity vulnerabilities

1 Low severity vulnerability

2 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
FrenlyFactory.sol User Functionalities: - Create one or more vaults with parameters - Deposit tokens to an existing vault (only creator) - Execute sell from vault (only influencer) - Claim ETH from OTC pool (only influencer) - Cancel OTC pool (only influencer) - Propose vault ownership transfer (only current influencer) - Accept vault ownership transfer (only proposed influencer) - Cancel vault ownership transfer (only current influencer) - Stake PartyX tokens in a reseller pool (only influencer) - Stake unstaked SPLASH to a reseller pool (only influencer) - Move SPLASH between reseller pools (only influencer) - Initiate PartyX unstake (only influencer) - Claim unstaked PartyX (only influencer) - Unstake SPLASH from multiple reseller pools (only influencer) - Unstake all SPLASH to the vault (only influencer) - Unstake all SPLASH and initiate PartyX unstake (only influencer) - View the next vault name for a base name	**Contract achieves this functionality.**

- View all vaults created by a creator - View all vaults associated with an influencer - View the total number of vaults - View the vault address by index - View if ENS is enabled - View vault ownership transfer status - View vault staking status - Check if the PoolParty staking pool is open - View the reseller pools a vault has staked to Admin Functionalities (UPGRADER_ROLE) - Authorize contract upgrade Owner Functionalities (DEFAULT_ADMIN_ROLE) - Update the vault implementation address - Update ENS node - Set PoolParty staking contract and PartyX token addresses	
FrenlyVault.sol User Functionalities: - View token price in ETH - View token price in USD - View the remaining initials available - View initial status - View if a proposed sale amount is allowed - View ownership transfer status - View SPLASH staked in a specific reseller pool - View staking status - View SPLASH staked in a specific reseller pool	**Contract achieves this functionality.**

Admin Functionalities (DEFAULT_ADMIN_ROLE) - Sell tokens or create an OTC pool - Claim ETH from the OTC pool - Cancel the OTC pool - Propose ownership transfer - Accept ownership transfer - Cancel the ownership transfer - Stake PartyX in the reseller pool - Stake unstaked SPLASH to the reseller pool - Move SPLASH between reseller pools - Initiate PartyX unstake - Claim unstaked PartyX - Unstake SPLASH from multiple pools - Unstake all SPLASH to the vault - Unstake all SPLASH and initiate PartyX unstake	

Code Quality

This audit scope involves the smart contracts of the Frenly project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation; however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Frenly project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] FrenlyFactory#claim - Permanent Denial of Service via Balance Manipulation

Description

The `claim` function in the Factory contract is vulnerable to a permanent Denial of Service attack. The vulnerability stems from the unsafe calculation of `ClaimedAmount` using the difference between the vault's balance after and before the claim.

Vulnerability Details

The `claim` function in the FrenlyFactory is responsible for orchestrating `ETH` payouts from a vault's Over-The-Counter (OTC) sales. It calculates the amount of `ETH` to be sent to the influencer by measuring the vault's balance before and after the `vaultContract.claim()` call. The core vulnerability lies in this flawed "balance delta" logic, combined with the fact that the vault's own `claim` function immediately sends its *entire* `ETH` balance to the influencer.

```
  function claim(address vault) external nonReentrant {

        if (!isValidVault[vault]) revert InvalidVault();

        IVault vaultContract = IVault(payable(vault));

        if (msg.sender != vaultContract.influencer()) revert OnlyInfluencer();

        address payable influencer = vaultContract.influencer();

         uint256 balanceBefore = address(vaultContract).balance;  // Before it will be 0
eth.

        address poolAddress =  vaultContract.claim(); // Upon calling claim() --> the ETH
that is received from PoolParty will be sent to Influencer

        uint256 claimedAmount = address(vaultContract).balance - balanceBefore; // @audit
[High] - Can Cause Permanent DOS  by sending 1 wei pre calling this claim().
```

```
        if (claimedAmount > 0) {

            Address.sendValue(influencer, claimedAmount);

            emit OTCEthClaimed(vault, influencer, claimedAmount, poolAddress);

        }

    }
```

An attacker can permanently break this process with a simple dusting attack:

1. Attack Preparation: An attacker sends a minuscule amount of `ETH` (e.g., `1 wei`) directly to a `FrenlyVault` address. The vault's balance is now `1 wei`.
2. Legitimate Claim Trigger: The influencer, expecting a payout from an OTC sale, calls `claim()` on the `FrenlyFactory`.
3. Flawed Execution Flow:
   a. `FrenlyFactory` records `balanceBefore = 1 wei`.
   b. `FrenlyFactory` calls `vaultContract.claim()`.
   c. Inside the vault, PoolParty sends the real OTC proceeds (e.g., 10 ETH) to the vault. The vault's balance becomes `10 ETH + 1 wei`.
   d. The vault's own logic immediately sends the *entire* `10 ETH + 1 wei` to the influencer. The vault's balance is now 0.
   e. Control returns to the `FrenlyFactory`.
4. Transaction Reversion: The factory attempts to calculate `uint256 claimedAmount = address(vaultContract).balance - balanceBefore;`, which becomes `0 - 1`. This calculation results in a subtraction underflow, causing the entire transaction to revert.

Impact

This vulnerability creates a permanent Denial of Service for a core protocol feature. Once a vault is "dusted," the `claim()` function for that vault will forever revert.

Permanent Loss of Functionality: The influencer is permanently unable to claim any current or future OTC proceeds through the protocol.

Locked Funds: The ETH generated from OTC sales will be trapped in the PoolParty contract, as the mechanism to claim them is broken.

Protocol Integrity Damage: This flaw undermines trust in the protocol's ability to safely manage user funds and execute its core promises.

Recommendation

The responsibility for calculating and sending payouts must be handled by a single contract. The balance-delta accounting pattern should be removed entirely.

```
// In FrenlyVault.sol's claim()

function claim() external returns (uint256 amountClaimed) {

    uint256 balanceBefore = address(this).balance;

    IPoolParty(poolPartyAddress).claimETH();

    amountClaimed = address(this).balance - balanceBefore;

    // Send ETH back to Factory

}



// In FrenlyFactory.sol's claim()

function claim(address vault) external nonReentrant {

    // ... (checks)

    uint256 claimedAmount = vaultContract.claim(); // Use the reliable return value



    if (claimedAmount > 0) {

        Address.sendValue(influencer, claimedAmount);

        emit OTCEthClaimed(vault, influencer, claimedAmount, poolAddress);

    }

}
```

Status

Resolved

[H-02] VaultDeploymentLib#executeBatchTransfers - Flawed Accounting with Fee-on-Transfer Tokens During Vault Creation

Description

The VaultDeploymentLib does not account for fee-on-transfer tokens when recording token amounts. When dealing with tokens that charge a fee on transfer (such as some deflationary tokens), the actual amount received by the contract is less than the amount specified in the transfer. The current implementation stores the full `amount` without considering that the actual received amount might be reduced by transfer fees, leading to accounting discrepancies and potential fund loss.

Vulnerability Details

The `executeBatchTransfers` function within VaultDeploymentLib is responsible for aggregating token deposits during batch vault creation. It calculates the total amount to be transferred for each token by summing up the `initialDeposit` values from the input parameters. However, it assumes that the amount specified in the safeTransferFrom call is the amount that actually arrives in the contract.

For fee-on-transfer (or deflationary) tokens, this assumption is false. A fee is taken during the transfer, so the receiving contract's balance increases by less than the specified amount. The library incorrectly records the pre-fee amount, creating a discrepancy between the contract's internal accounting and its actual token balance.

Impact

This accounting error introduces a critical bug that can render user funds inaccessible.

Systemic Insolvency: The protocol will consistently believe it holds more tokens than it actually does. Over time, this deficit can grow to a significant level.

Reverted Withdrawals: Any future logic that relies on this flawed accounting to process withdrawals or transfers will likely fail. For instance, an attempt to withdraw the "full" account amount will revert with an "insufficient balance" error.

Loss of Funds: Users may be permanently unable to retrieve the portion of their funds represented by the accounting discrepancy.

Recommendation

To handle fee-on-transfer tokens correctly, the system must always calculate the amount received by checking the balance change, rather than trusting the input amount.

```
// In VaultDeploymentLib.sol

function executeBatchTransfers(/*...*/) internal {

    // ...

    for (uint256 i = 0; i < uniqueTokens; i++) {

        if (amounts[i] > 0) {

            uint256 balanceBefore = IERC20(tokens[i]).balanceOf(target);

            IERC20(tokens[i]).safeTransferFrom(sender, target, amounts[i]);

            uint256 actualReceived = IERC20(tokens[i]).balanceOf(target) - balanceBefore;



            // Important: All subsequent logic in this function and the calling

            // `createVault` function must be refactored to use `actualReceived`

            // instead of the original `amounts[i]` or `params[i].initialDeposit`

            // for token accounting and event emission.

        }

    }

}
```

Note: The above recommendation is a starting point. The fix requires refactoring any subsequent logic that relies on `params[i].initialDeposit` after this transfer, ensuring it uses the post-fee `actualReceived` value.

Status

Resolved

Medium

[M-01] FrenlyVault#getEthUSDPrice - Use of Stale Price Data from Chainlink Oracle

Description

The `getEthUsdPrice` function retrieves the latest price from a Chainlink oracle but fails to check if the data is stale. An oracle may stop updating due to network congestion, faulty node operators, or extreme market conditions. Relying on this potentially outdated price can lead to incorrect valuation of assets.

Impact

All financial calculations that depend on this price, such as enforcing USD-based sell limits, would be incorrect. This could lead to users bypassing intended limits or executing trades at unfair effective prices.

Recommendation

Implement a staleness check by validating the updatedAt timestamp from `latestRoundData()` against a reasonable threshold (e.g., 1-2 hours). Revert the transaction if the data is too old.

```
function getEthUsdPrice() internal view returns (uint256 price) {

                (    ,    int256    _price,    ,    uint256    updatedAt,    )    =
IAggregatorV3(ethUsdPriceFeed).latestRoundData();

    // Stale price check

    if (block.timestamp - updatedAt > 3600) revert("Price data is stale");

    if (_price <= 0) revert("Invalid price");

    // ... (rest of the logic)

}
```

Status

Resolved

[M-02] FrenlyFactory - Risk of Permanently Ownerless Contract

Description

The contract inherits from `AccessControlUpgradeable` but doesn't override the `renounceRole` function, which could allow administrators to accidentally renounce critical roles, potentially leaving the contract without proper access control.

Impact

If the `DEFAULT_ADMIN_ROLE` is renounced, the contract could become permanently ownerless, making critical administrative functions inaccessible.

Recommendation

Override the `renounceRole` function to prevent renouncing critical roles:

```
function renounceRole(bytes32 role, address account) public virtual override {
    require(role != DEFAULT_ADMIN_ROLE, "Cannot renounce admin role");
    super.renounceRole(role, account);
}
```

Status

Resolved

[M-03] VaultDeploymentLib&Vault - Inconsistent Limit Configuration Logic

Description

When a vault is configured with `useFixedLimits = true`, the `fixedDailySellLimit` (in token units) and the `dailySellLimitUsd` (in USD) are set as independent parameters. There is no validation to ensure these values are rationally related. A user could set a very high `fixedDailySellLimit` alongside a very low `dailySellLimitUsd`, creating confusing and potentially unsafe configurations.

Impact

This could lead to situations where the actual `USD` value of token limits significantly differs from intended `USD` limits due to price fluctuations, potentially bypassing intended restrictions.

Recommendation

Implement dynamic calculation or validation logic:

```
function validateLimits(IVault.LimitConfig memory limits, uint256 tokenPriceUsd) internal
pure {
    if (!limits.useFixedLimits) return;

    uint256 calculatedDailyLimit = (limits.dailySellLimitUsd * 1e18) / tokenPriceUsd;
    uint256 calculatedTxLimit = (limits.transactionSellLimitUsd * 1e18) / tokenPriceUsd;

    require(
        limits.fixedDailySellLimit <= calculatedDailyLimit * 110 / 100, // 10% tolerance
        "Fixed daily limit exceeds USD equivalent"
    );
    require(
        limits.fixedTransactionSellLimit <= calculatedTxLimit * 110 / 100,
```

```
        "Fixed transaction limit exceeds USD equivalent"
    );
}
// PSEUDO CODE ONLY, DEVS SHOULD IMPLEMENT CORRECTLY FOLLOWED BY AUDITORS VERIFICATION
```

Status

Acknowledged

[M-04] FrenlyFactory&FrenlyVault#sell - Blind Execution of User-Provided Swap Data

Description

The `sell()` function in both the factory and vault is designed to execute a token swap on an external DEX. It does this by accepting a pre-packaged bytes calldata `swapData` from the caller (the influencer) and executing it via a low-level call: `dexAddress.call(swapData)`.

The fundamental flaw is that the contract places complete trust in the integrity of this `swapData`. There is no validation to ensure the data performs the intended action, namely swapping the vault's token for another and sending the proceeds to the influencer. A malicious or sophisticated influencer can craft `swapData` to execute arbitrary actions on the DEX contract.

Example Attack Vectors:

1. Approval Griefing: The influencer could craft swapData to call approve() on the DEX aggregator, granting spending approval for the vault's tokens to an arbitrary address. While this doesn't directly steal funds, it creates a lingering, dangerous approval that could be exploited later if another vulnerability is found.
2. Value Extraction via Malicious Routing: The influencer could create a DEX pool with extremely poor liquidity or a malicious routing contract. They could then provide `swapData` that forces the trade through this path, resulting in massive slippage. The vault's tokens would be swapped for a fraction of their market value. The influencer still receives the proceeds, but they have effectively burned a significant portion of the creator's assets in the process.
3. Complex Calls: The swapData could encode calls to other unrelated functions on the DEX contract, potentially triggering unexpected states or interacting with other systems.

Impact

While the influencer cannot directly steal tokens from the vault (as the vault holds the tokens, not the influencer's EOA), they can abuse the trust placed in them to cause value loss or create risky states.

- Asset Value Loss: The creator's assets held in the vault can be drained inefficiently, violating the spirit of the collaboration.
- Protocol Risk: The contract could be made to interact with unintended external contracts or create dangerous approvals, increasing the system's attack surface.
- Denial of Service: Crafted swapData could cause transactions to revert in confusing ways, making the sell function difficult to use.

Recommendation

Do not accept raw, arbitrary `swapData`. The contract should build the swap data itself based on high-level, trusted parameters.

Refactor the `sell()` function to accept parameters that define the swap's intent, such as:

`destinationToken`: The token to receive.

`minAmountOut`: The minimum amount of destinationToken to accept (to protect against slippage).

`recipient`: This should be hardcoded inside the vault to always be the influencer.

The FrenlyVault contract would then be responsible for ABI-encoding these parameters into the correct `swapData` format for the trusted `dexAddress`. This approach removes the ability for the caller to execute arbitrary code and ensures all swaps are performed safely and as intended.

Status

Resolved

Low

[L-01] FrenlyFactory#updateVaultImplementation - Implementation Address Not Validated as a Contract

Description

The `updateVaultImplementation` function does not check if the `_newImplementation` address contains code. An administrator could mistakenly set an Externally Owned Account (EOA) as the implementation address.

All subsequent calls to createVault would `fail` because the clone operation would target a non-contract address. This would effectively halt the creation of new vaults until a new, valid implementation is set.

Recommendation

Add a contract existence check using `_newImplementation`.code.length > 0.

Status

Resolved

QA

[Q-01] Contracts - Use of Floating Pragma

Description

The use of a floating pragma (e.g., ^0.8.20) means the contract can be compiled with different compiler versions, potentially introducing unintended bugs or behaviors that were not present during the audit.

Decreased determinism in the build process, which could complicate verification and production deployments.

Recommendation

Lock the pragma to a single, specific version for all production contracts (e.g., pragma solidity 0.8.20;).

Status

Resolved

[Q-02] FrenlyFactory#depositToVault - Redundant depositToVault Function

Description

The `depositToVault` function provides no additional functionality beyond direct token transfers, potentially creating confusion about the proper way to deposit tokens.

Users might assume this function provides additional security or validation that direct transfers don't, leading to inconsistent usage patterns and potential integration issues.

Recommendation

Either enhance the function with meaningful validation/features or remove it entirely

Status

Resolved

Centralisation

The Frenly project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Frenly project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au